

第五章 计算思维与算法

尹江津	信息学院
Q Q:	1961015633
微 信:	15674996078

目录

1. 计算思维

2. 问题与问题求解

3. 算法

1. 计算思维

1.1 三种基本的科学思维

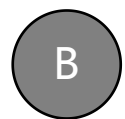
1.2 计算思维的定义

1.3 计算思维特征

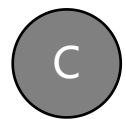
假设你面前有三扇关闭了的门，其中一扇的后面有一辆汽车，另外两扇门后面则什么都没有（即**空门**），选中后面有汽车的那扇门可赢得该汽车。当你选定了一扇门，系统会开启剩下的两扇门中的一扇**空门**；并问 “**要不要换选另一扇未打开的门？**”



换



不换



遁入空门



Java知己

提交

1.1 三种基本的科学思维

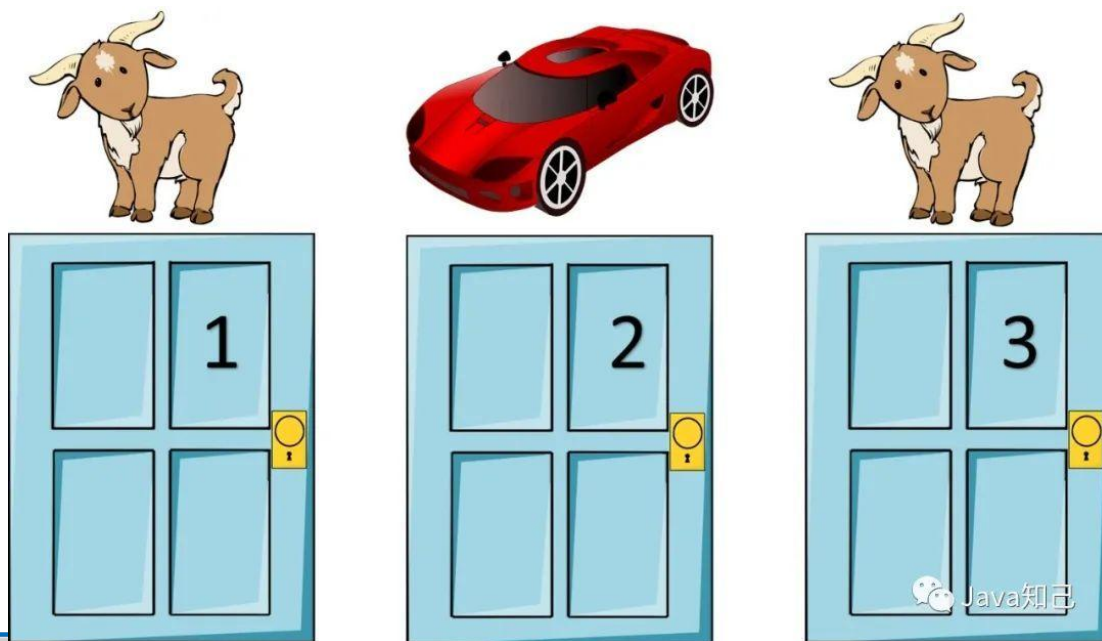
三门问题

- **三门问题**也称为**蒙提霍尔问题**，出自蒙提·霍尔（Monty Hall）主持的美国电视游戏节目《Let's Make a Deal》。参赛者会看见三扇关闭了的门，其中一扇的后面有一辆汽车，另外两扇门后面则各藏有一只山羊，选中后面有汽车的那扇门可赢得该汽车。当参赛者选定了一扇门，主持人会开启剩下的两扇门中有山羊的门；并问：“要不要换一扇门？”

答：

不换门获得汽车概率为 $1/3$

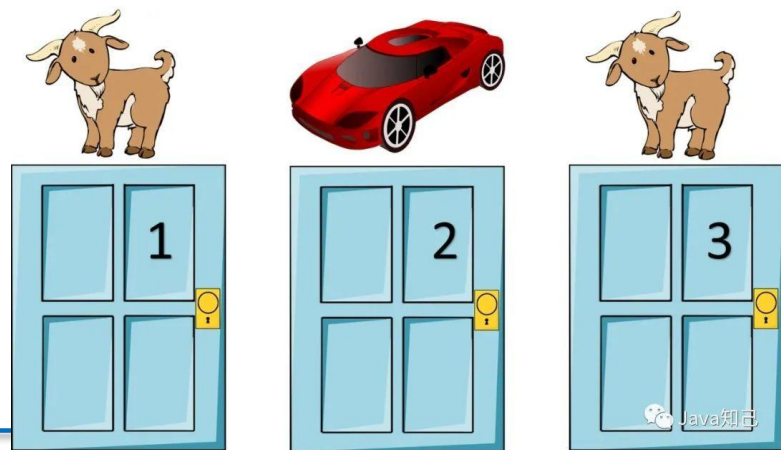
换门获得汽车的概率为 $2/3$



1.1 三种基本的科学思维

三门问题-悖论

- 有人说当排除掉B门后，剩余的两扇门应该概率各50%才对。选A或选C，应该是一样的。
- 有人说，假设有一个迟到后入场，让他随机选剩余的A或C，他的概率肯定是1/2。难道对于参赛者的概率是1/3和2/3，对于后者却是1/2和1/2，这不违背概率的客观性吗？
- 有人说，有两个参赛者，一个选A，一个选C，中间空门B打开。如果更改选择会提升概率，那么到底是A更好，还是C更好呢？这不自相矛盾吗？



1.1 三种基本的科学思维

1 三门问题

实验思维，以观察和总结自然规律为特征。

解决方案1—实验方式：

- 线上或者线下组织多人参与“三门问题”游戏，记录他们在主持人揭示信息后是否改变选择以及他们最终赢得奖品的情况，收集大量用户数据。
- 通过统计大量的样本数据，得出参与者坚持不换门和改变最初选择时，赢得汽车的概率。



1.1 三种基本的科学思维

2. 三门问题

解决方案2：数学演绎推理

- 假设，参与者开始选的是1号门，主持人打开的是2号门，这里需要求解的是：在主持人打开的是2号门的前提下，1号门有车 and 3号门车的概率分别是多少？

✓ 令事件A表示主持人打开2号门， B_i ($i=1,2,3$)表示第*i*号门有车。

$$P1 = P(1号门有车) * P(主持人打开2号门 | 1号门有车) = P(B1) P(A|B1) = 1/3 * 1/2 = 1/6$$

$$P2 = P(2号门有车) * P(主持人打开2号门 | 2号门有车) = P(B2) P(A|B2) = 1/3 * 0 = 0$$

$$P3 = P(3号门有车) * P(主持人打开2号门 | 3号门有车) = P(B3) P(A|B3) = 1/3 * 1 = 1/3$$

✓ 根据**贝叶斯定理**,

$$P(B1 | A) = P(1号门有车 | 主持人打开2号门) = P1 / (P1 + P2 + P3) = 1/6 \div 1/2 = 1/3$$

$$P(B3 | A) = P(3号门有车 | 主持人打开2号门) = P3 / (P1 + P2 + P3) = 1/3 \div 1/2 = 2/3$$

- 由此可见，应当换门，即从选择1号门，更换成选择3号门。

$$P(B_i|A) = \frac{P(AB_i)}{P(A)} = \frac{P(A|B_i)P(B_i)}{\sum_k P(A|B_k)P(B_k)}$$

理论思维，以推理演绎为特征。

1.1 三种基本的科学思维

3. 三门问题

解决方案3：编写程序模拟

通过编写程序模拟三门问题实际的决策过程，通过大量模拟试验，统计不同策略下的胜率来得出结论。

- ✓ 设定一个循环结构，模拟多次游戏 → ***for*循环**
- ✓ 定义三扇门分别存放车，羊，羊 → **列表或数组存储**
- ✓ 参赛者随机选择一扇门作为初始选择 → **random模块**
- ✓ 主持人揭示未被选中且没有羊的一扇门 → **删除列表或数组中满足条件的某个元素**

计算思维，以设计和构造为特征。

1.1 三种基本的科学思维

3. 三门问题

```
from random import choice
def stay():
    doors = ['car', 'goat', 'goat']
    choose = choice(doors)
    if choose == 'car':
        return 'win'
    else:
        return 'lose'
def switch():
    doors = ['car', 'goat', 'goat']
    choose = choice(doors)
    doors.remove(choose)
    doors.remove('goat')
    if doors == ['car']:
        return 'win'
    else:
        return 'lose'

if __name__ == '__main__':
    total = 100000
    win_switch = 0
    win_stay = 0
    for i in range(total):
        if switch() == 'win':
            win_switch += 1
        if stay() == 'win':
            win_stay += 1
    print('总的模拟次数:', total)
    print('换门后得到汽车:', win_switch, '胜率:', '%.2f%%'%(100*win_switch/total))
    print('不换门后得到汽车:', win_stay, '胜率:', '%.2f%%'%(100*win_stay/total))
```

模拟次数	换门胜率次数/概率		不换门胜率次数/概率	
1000	650	65.00%	354	35.40%
10,000	6645	66.45%	3363	33.63%
100,000	66563	66.56%	33335	33.34%
10,000,000	666367	66.64%	333687	33.37%
100,000,000	6664692	66.65%	3333822	33.34%

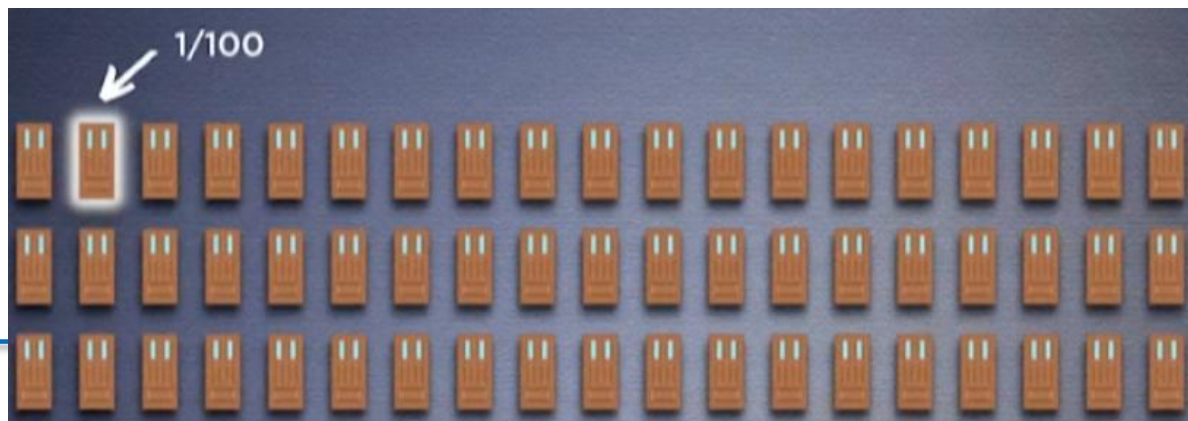
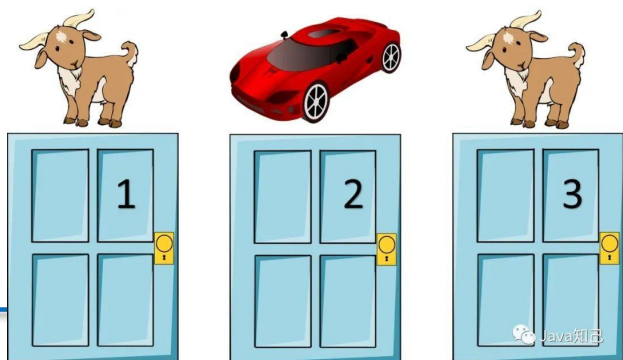
结论：大量模拟实验中，换门获得汽车概率无限接近2/3，不换获得汽车概率无限接近1/3。

1.1 三种基本的科学思维

三门问题-悖论

- 有人说，当排除掉B门后，剩余的两扇门应该概率各50%才对。选A或选C，应该是一样的。（答：有两个可能状态，未必是平权的）
- 有人说，假设有一个迟到后入场，让他随机选剩余的A或C，他的概率肯定是1/2。难道对于参赛者的概率是1/3和2/3，对于后者却是1/2和1/2，这不违背概率的客观性吗？（答：不同的人，获取不同的信息，胜率当然不一样的）
- 有人说，有两个参赛者，一个选A，一个选C，中间空门B打开。如果更改选择会提升概率，那么到底是A更好，还是C更好呢？这不自相矛盾吗？

（答：前提是B门必须是山羊，因此成功概率均为1/2。PS：若B为汽车游戏中止。）



1.1 三种基本的科学思维

三门问题



1.1 三种基本的科学思维

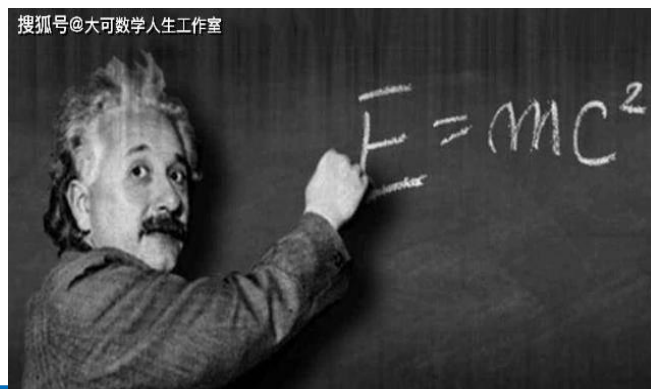
三种基本的科学思维方式

科学思维：是指经过感性阶段获取的大量材料通过整理和改造，形成概念、判断和推理，以便反映事物的本质和规律。即理性认识及其过程。主要包含以下三种：

- **实验思维**又称实证思维，以观察和总结自然规律为特征，以物理学科为代表。（**强调归纳**）
- **理论思维**又称推理思维，以推理和演绎为特征，以数学学科为代表。（**强调推理**）
- **计算思维**又称构造思维，以设计和构造为特征，以计算机学科为代表。（**强调能自动求解**）



伽利略比萨斜塔实验



爱因斯坦的质能方程



DeepMind公司的AlphaGo机器

1.2 计算思维的定义

1. 计算思维的定义

- 2006年3月，美国卡内基·梅隆大学计算机科学系主任周以真教授（Jeannette M. Wing）首次明确提出“计算思维”的概念，她提到计算思维如同所有人都具备“**读、写、算**”（简称3R）能力一样，成为适合于每个人的一种普遍的认识和一类普适的技能。
- 计算思维的定义：**计算思维是运用计算机科学的基础概念去求解问题、设计系统和理解人类行为，它涵盖了计算机科学的一系列思维活动。**

Wing J M. **Computational thinking**[J]. *Communications of the ACM*, 2006, 49(3): 33-35. 引用次数：[16700次](#)

Computational Thinking

It represents a universally applicable attitude and skill set everyone, not just computer scientists, would be eager to learn and use.



Computational thinking builds on the power and limits of computing processes, whether they are executed by a human or by a machine. Computational methods and models give us the courage to solve problems and design systems that no one of us would be capable of tackling alone. Computational thinking confronts the riddle of machine intelligence: What can humans do better than computers? and What can computers do better than humans? Most

cisely. Stating the difficulty of a problem accounts for the underlying power of the machine—the computing device that will run the solution. We must consider the machine's instruction set, its resource constraints, and its operating environment.

In solving a problem efficiently, we might further ask whether an approximate solution is good enough, whether we can use randomization to our advantage, and whether false positives or false negatives are allowed. Computational thinking is reformulating a seemingly difficult problem into one we know how to solve, perhaps by reduction, embedding, transformation, or simulation.

思维活动是指人脑通过**感知、记忆、思考、判断、推理**等一系列过程，对客观事物或情境进行认知和理解的心理活动。

1.2 计算思维的定义

周以真 (Jeannette M. Wing) 简介

- 1979年6月获得麻省理工学院学士学位;
- 1983年6月获得麻省理工学院博士学位;
- 1983年—1985年任南加利福尼亚大学计算机科学系助理教授;
- 1985年起任教于卡内基梅隆大学;
- 1999年—2004年任卡内基梅隆大学计算机科学学院学术副院长;
- 2004年—2007年任**卡内基梅隆大学**计算机科学系主任;
- 2007年—2012年任微软-卡内基梅隆**计算思维中心主任**;
- 2013年—2017年任微软全球副总裁;
- 2017年任哥伦比亚大学计算机科学教授;
- 2021年9月任哥伦比亚大学常务副校长;
- 2024年2月当选为**美国国家工程院院士**。



1.3 计算思维的特征

计算思维的本质

抽象



自动化

抽象 (Abstraction) :

- 抽象是指从复杂的真实世界中**提取出关键特征**，忽略不必要的细节，构建简洁明了的模型或框架。比如，用数据结构（如列表、集合、树等）来表示复杂的数据关系，用算法来描述解决问题的步骤和逻辑，而不必关注实现这些结构和算法的具体硬件细节。

自动化 (Automation) :

- 自动化在计算思维中指的是设计和**应用算法、程序和系统来自动执行任务或解决问题的过程**。通过编写程序，计算机可以按照预定的逻辑自动完成一系列操作，无需人工干预。

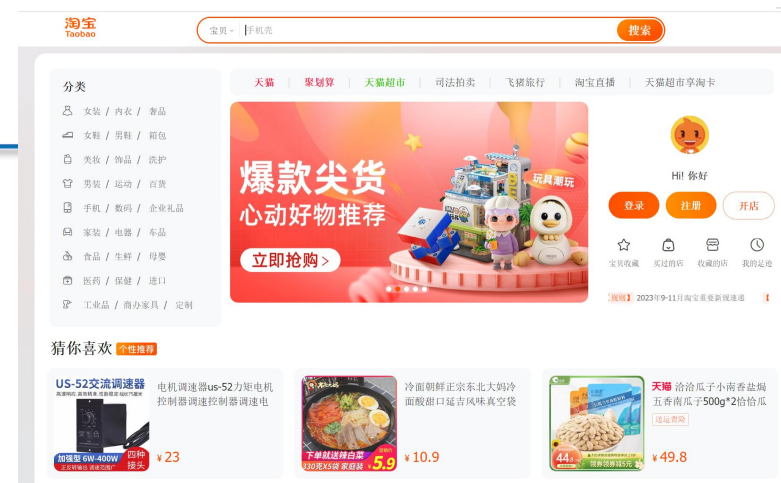
总的来说，**抽象和自动化在计算思维中相辅相成**，抽象帮助我们理解和描述问题，而自动化则实现了对问题解决方案的执行和扩展。

1.2 计算思维的定义

系统设计中的计算思维

比如设计一个在线购物系统

- **抽象化**：抽象出系统的主要组成部分，如用户账户管理、商品目录管理、订单管理等
- **问题分解**：将系统设计分解成多个子系统，如用户界面设计（前端）、服务器端应用程序设计（后端）等。
- **算法设计**：设计搜索算法以使用户快速查找商品。基于用户的购物历史和行为数据为其提供个性化的推荐。
- **数据结构**：使用合适的数据结构来存储商品信息、用户信息、订单信息等，如关系型数据库、文档数据库或图形数据库等。



1.2 计算思维的定义

近十几年来，随着对于计算思维理论的深入研究，逐步由1.0版本发展到2.0版本。

- 计算机思维1.0更偏重于基础计算与编程。
- 计算机思维2.0则更加强调跨学科的综合应用、智能系统设计和解决实际问题的能力，体现了更高的抽象层次和更广泛的适用范围。

计算思维1.0	计算思维2.0
计算思维是计算机科学家的思维模式	计算思维是所有领域的科学家在应用计算和计算模型时所采取的思维模式
计算思维是从计算机科学中产生，逐步扩散到其他科学领域	每个科学领域都有属于自身特点的计算思维，贯穿通过计算模型求解的设计、执行和评价过程
学习编程能力过程中产生计算思维，计算思维主要是大量的编程技巧	计算思维提供了有关模型和算法的丰富概念，学习和理解计算思维用以产生科学的编程能力
算法必须与程序紧密联系	算法并不一定要联系程序和计算机，算法是信息处理的技术，一些生物体内部进行的自然计算也属于算法
实施算法设计要求具备计算机科学的领域知识，特别是关于计算机的基本知识	每个人都可以在本领域找到问题计算求解的模型、算法和操作，并不需要图灵机这样的通用概念
应该关心程序背后的计算机概念而不是程序本身	应该关心程序背后反映领域特点的内容，注重本学科解决问题的计算化趋势

1.3 计算思维的特征



五大特征

- **概念化，不是程序化**
- **是基础的，不是机械的技能**
- **是人的，不是计算机的思维方式**
- **数学和工程思维的互补与融合**
- **是思想，不是人造物**

1.3 计算思维的特征



(1) 概念化，不是程序化。

计算机科学不是计算机编程。像计算机科学家那样去思维意味着远远不止能为计算机编程，它还要求能够在抽象的多个层次上思维。

1. 计算思维**不仅仅是**编写程序或代码，**而更是一种更高层次的思考方式**，它涉及到对问题的抽象化理解、逻辑分析和系统设计。计算思维**不局限于具体的编程语言或代码实现**。
2. 从高层次的抽象概念设计到低层次的具体实现细节，都需要具备良好的计算思维能力。

1.3 计算思维的特征

(2) 基础的，不是机械的技能。

基础的技能是每一个人为了在现代社会中发挥职能所必须掌握的。生搬硬套之机械的技能意味着机械的重复。

(3) 人的，不是计算机的思维。

计算思维是人类求解问题的一条途径，是人的思想和方法。计算机枯燥烦闷，人类聪颖且富有想象力，人类赋予计算机以激情。配置了计算设备，就能用自己的智慧去解决那些计算时代之前不敢尝试的问题。计算机之所以能够求解问题，是因为人将计算思维的思想赋予了计算机，计算机才能够进行如迭代、递归等计算。

1.3 计算思维的特征

(4) 数学和工程思维的互补与融合。

计算思维整合了数学逻辑的严谨性和工程设计的实用导向性。它**借鉴了数学中的抽象和形式化方法，同时也吸收了工程思维中的系统化和优化理念**，将二者相结合，以解决现实世界中复杂的计算和信息处理问题。



(5) 是思想，不是人造品。

计算思维是一种思维模式和方法论，而非具体的物质产品或技术。它是一种无形的智力资产，存在于人们的头脑中，通过教育和实践得以培养和发展。相比于具体的计算机程序或软件，计算思维更加关注于思维过程本身，是驱动技术创新和应用发展的源泉。

计算思维是运用（ ）的基础概念去求解问题、设计系统和理解人类行为，它涵盖了计算机科学的一系列思维活动。

- ☐ A. 现代科学
- ☒ B. 计算机科学
- ☐ C. 思维科学
- ☐ D. 计算科学

选择题

下面有关计算思维的描述中，正确的是（ B ）

- A. 计算思维是计算机的思维。
- B. 计算思维是人的思维。
- C. A和B均正确
- D. A和B均不正确



（3）人的，不是计算机的思维。

计算思维是人类求解问题的一条途径，是人的思想和方法。计算机枯燥烦闷，人类聪颖且富有想象力，人类赋予计算机以激情。

2. 问题与问题求解

2.1 问题及其形式化描述

2.2 问题求解的一般过程

2.3 农机调度案例问题求解

5.2 问题与问题求解

1. 问题及形式化描述

人们的活动都是一个不断提出问题、发现问题和解决问题的过程。那么何为问题？所谓**问题（Problem）**，就是**不能用已有的知识经验直接加以处理并因此而感到疑难的情境**。

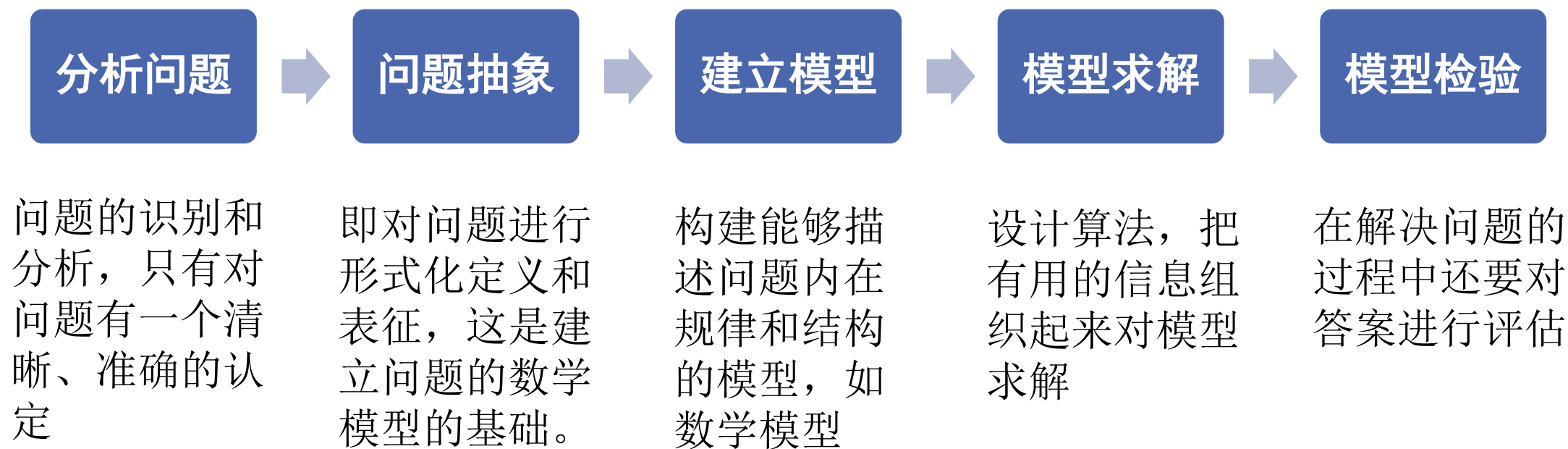
问题一般具有3个基本成分：

- （1）**起始状态**：一组已知的关于问题条件的描述，即问题的初始条件、背景信息；它是问题求解的起点。
 - （2）**目标状态**：解决问题时希望达到的理想状态或期望结果，它是问题解决过程中的终点，也即问题要求的答案；
 - （3）**障碍或限制条件**：从起始状态到达目标状态的过程中，可能会遇到的一系列阻碍或限制条件，必须在问题解决过程中予以考虑和克服。
-

5.2 问题与问题求解

2. 问题求解的一般过程

问题求解的过程就是探寻得到问题答案或达到问题目标的过程和方法，是从问题的**初始状态**到**目标状态**的演化过程。



5.2 问题与问题求解

3 农机调度问题示例

(1) 问题描述:

我国农机实行跨区作业服务模式，农户希望农田能及时颗粒归仓，机手希望在有限的时间内为尽可能多的农户服务，从而得到更多的收益。因此需要机手需要综合**农田位置、面积、距离**等多种因素，以作业收益最大为目标，来求解最佳的农机调配规划方案，以实现农机作业收益最大化问题。



5.2 问题与问题求解

3 农机调度问题示例

(2) 问题抽象:

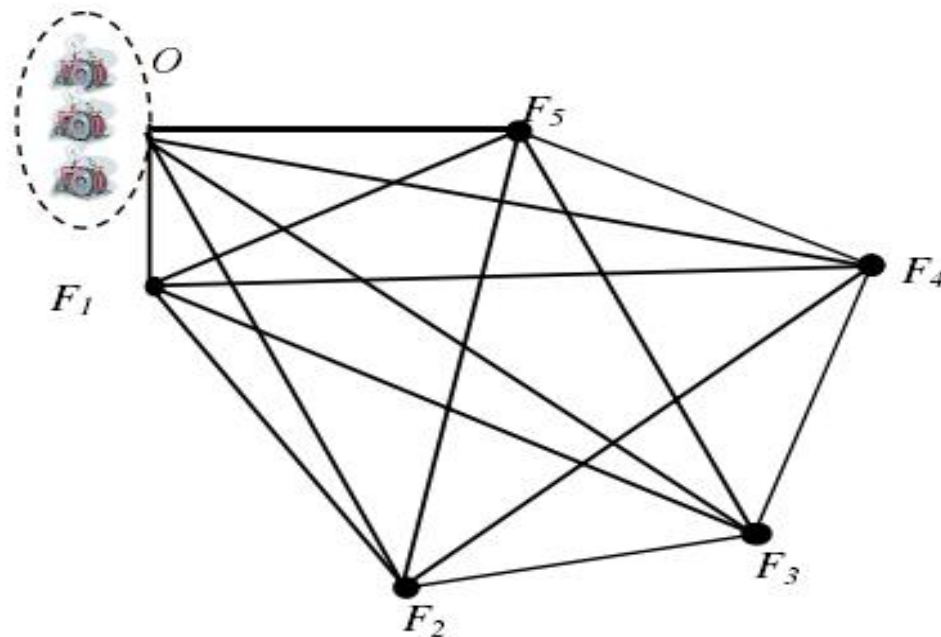
如图所示，节点O为机主的农机资源所在的位置，节点F1-F5表示机主所选的各个农田作业点的位置。每个农田作业点的面积、作业价格、所在位置、适宜服务的时间范围以及图中任意两点间的距离已知。

(3) 问题建模:

建立最优化数学模型:

$$\text{MaxP} = \sum_{j=1}^n p_j A_j - \sum_{j=0}^n T_{i,j} - \sum_{j=1}^n w_j$$

- p_j 表示对于农田块j 单位面积可得到的收入
- A_j 表示农田块j 的待作业面积
- $T_{i,j}$ 表示由地点i 到j 的运输成本
- w_j 表示农田块j 的作业成本

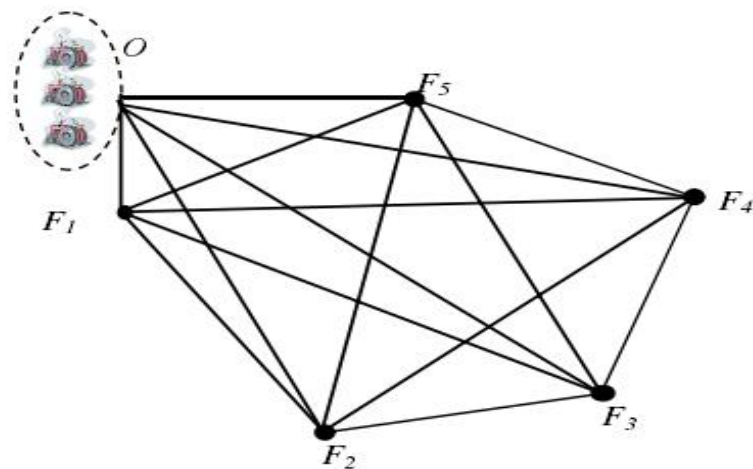


5.2 问题与问题求解

(4) 建模求解：

数学模型：

$$\text{MaxP} = \sum_{j=1}^n p_j A_j - \sum_{j=0}^n T_{i,j} - \sum_{j=1}^n w_j$$



单机作业调度问题：可以近似为旅行商问题（TSP），利用贪心算法、动态规划、遗传算法、模拟退火算法等。

多机作业调度问题：问题更复杂，需要考虑多台农机的协同作业，同时考虑农机之间的通信等。考虑利用分层规划法、粒子群优化算法、蚁群算法、多智能体强化学习算法等

问题求解的方法——算法

3. 算法

3.1 算法的定义

3.2 算法的特征

3.3 算法的表示方法

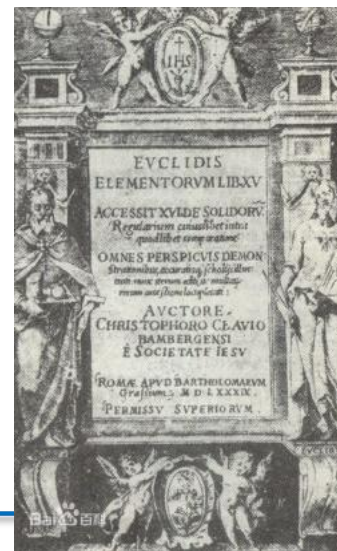
3.4 常用算法

5.3 算法

“算法”一词出现在唐代，当时有《算法》专著。宋代出现了《算法绪论》、《算法秘诀》等，元代出现了《丁巨算法》，明代出现了程大位《算法统宗》，清代出现了《开平算法》、《算法一得》、《算法全书》等。其中具有代表性的是宋代数学家**杨辉**的《**杨辉算法**》。



Algorithm “算法”原为algorism，意思是阿拉伯数字的运算法则，在18世纪演变为“algorithm”。史上第一个算法是欧几里得算法，即**辗转相除法**。古希腊数学家**欧几里得**在公元前3世纪在其著作《**几何原本**》第七卷中阐述了这个算法。



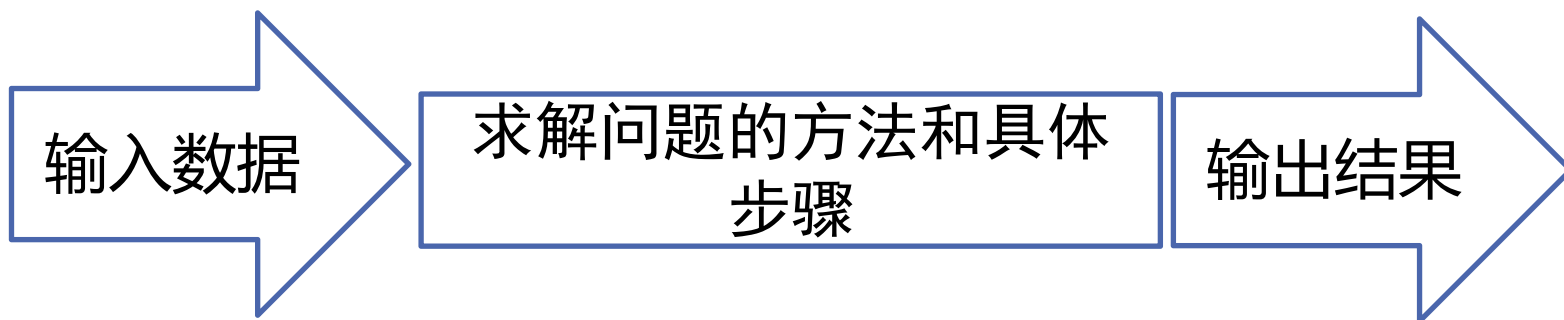
5.3 算法

1 算法的定义

算法是一种用于解决特定问题或执行特定任务的一系列明确、有限的步骤。它是一种有序的、有限长的操作序列，通常用于计算、数据处理、自动推理、问题求解等场合。

Q:什么是算法

通俗地讲，**算法(Algorithm)是步骤和描述**



5.3 算法

1 算法的定义

Q:什么是算法? **算法(Algorithm)是步骤和描述**

【例】交换酱油和醋两瓶调料。

问题描述：有两瓶调料，分别标明的是“酱油”和“醋”。但由于工人疏忽，将瓶子灌装错了，现在要求将其交换回来。

解决思路：设目前“酱油”瓶为1号瓶，它里面装的实际是醋；设目前“醋”瓶为2号瓶，它里面装的实际是酱油；设空瓶为3号瓶。

算法描述：

S1：将1号瓶中的醋倒入3号瓶

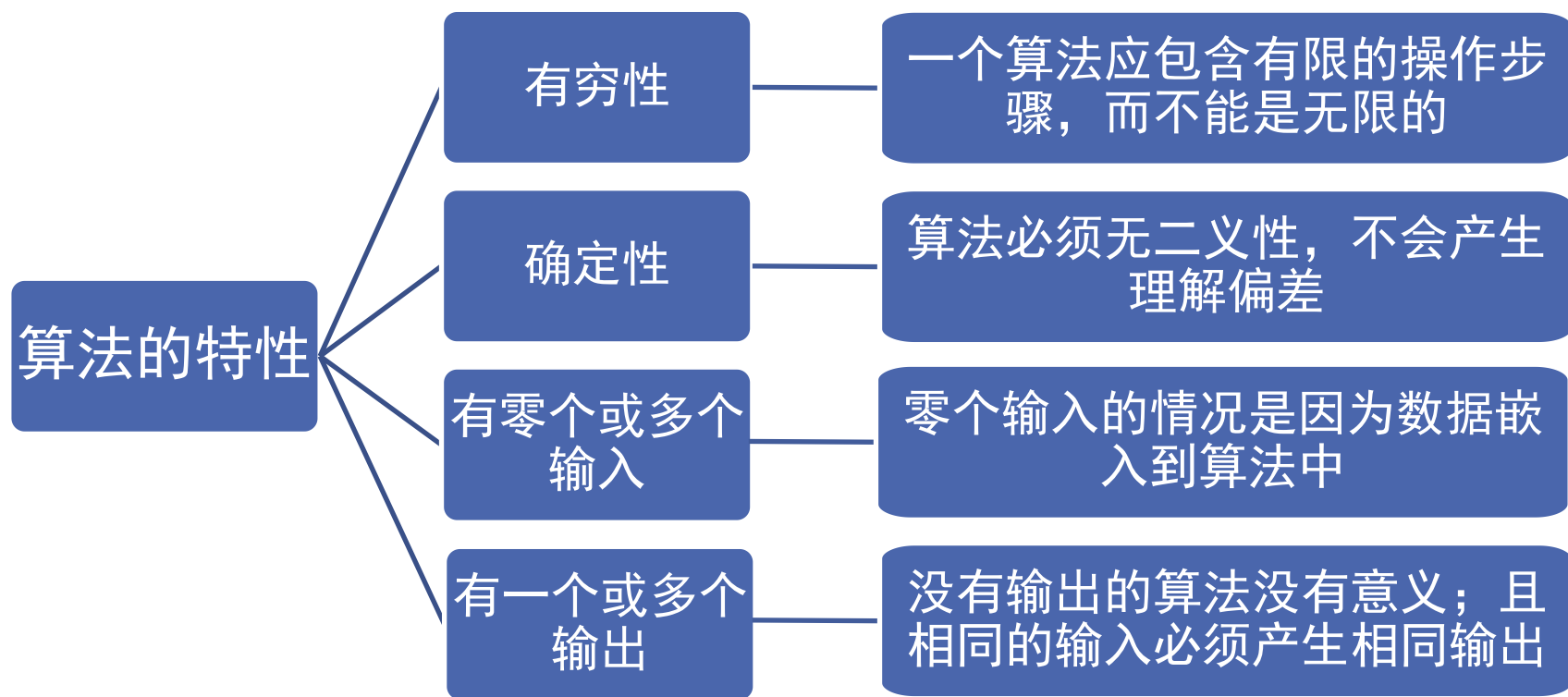
S2：将2号瓶中的酱油倒入1号瓶

S3：将3号瓶中的醋倒入2号瓶

5.3 算法

2 算法的特征

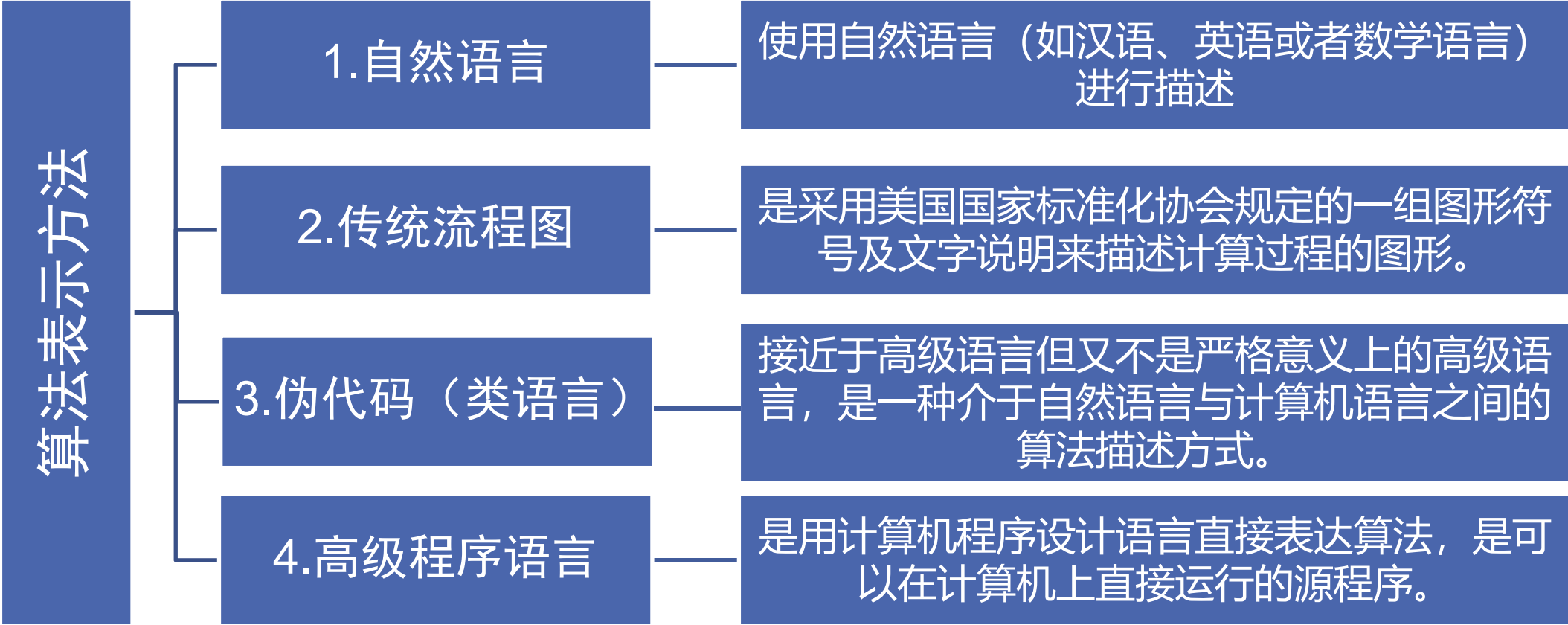
算法四大特征：**有穷性**、**确定性**、**零个或多个输入**和**一个或多个输出**。



5.3 算法

3. 算法表示方法

一个问题可以设计不同的算法来求解，而同一个算法可以采用不同的形式来表述，常用的**描述方法有以下4种**。



5.3 算法

3. 算法表示方法—自然语言

自然语言表示算法就是把算法的各个步骤，依次用人们熟悉的自然语言表示出来。

优点：通俗易懂，容易掌握。

缺点：语句繁琐，文字冗长，难以清楚表达复杂算法的逻辑，容易出现歧义性。

【例】用自然语言描述计算并输出 $z=x \div y$ 的流程：

- (1) 输入变量 x, y ;
- (2) 判断 y 是否为0;
- (3) 如果 $y=0$ ，则输出“出错提示”信息;
- (4) 否则计算 $z=x/y$;
- (5) 输出 z 。

◆ 无鸡鸭也可无鱼肉也可青菜豆腐不可少
◆ 我赞成他也赞成你怎么样

5.3 算法

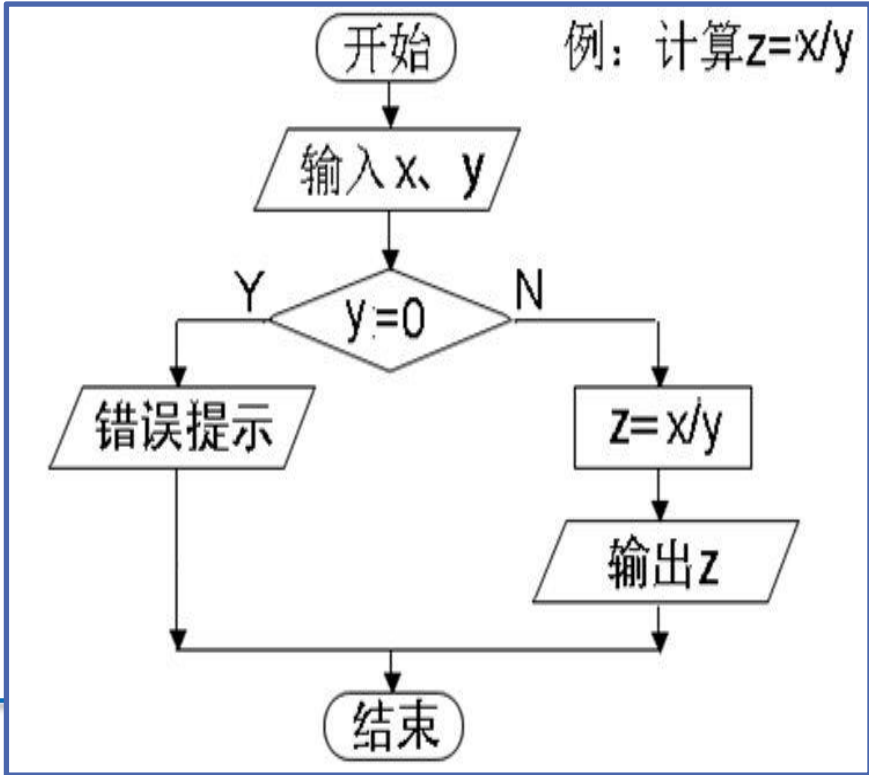
3. 算法表示方法—流程图表示

流程图是采用美国国家标准协会规定的一组图形符号及文字说明来描述算法的图形。流程图可以**直观**地表示算法的整个结构，**易于理解，便于检查修改**，但不方便表达数据处理过程。

流程图通常较好地表示**顺序、分支、循环等控制流**，但对于**复杂的数据结构（如链表、树、图等）和数据之间的深层次关系**，**流程图难以直观地展示**

图示	名称	说明
	开始/结束框	算法的开始或结束
	输入/输出框	变量的输入或输出
	处理框	变量的计算与赋值
	判断框	条件判断
	流程线	算法的流向
	连接点	算法的连接

【例】用传统流程图表示：输入x、y，计算 $z=x\div y$ ，输出z。



5.3 算法

3. 算法表示方法—伪代码表示

伪代码也称类语言，是一种介于自然语言与计算机语言之间的算法描述方式。伪代码具有高级语言的一般语句结构，撇掉语言中的细节，把注意力集中在算法处理步骤本身的描述上。**伪代码结构性强，容易书写和理解，不拘泥于特定语言的语法结构**，是在实际工程中一种**最常用的**算法表示方式。

【例】用伪代码描述：输入 x 、 y ，计算 $z=x \div y$ ，输出 z 。

```
(1) Begin          /* 算法伪代码开始 */
(2) 输入  $x$ ,  $y$       /* 输入变量 $x$ 、 $y$  */
(3) if  $y=0$  then Error 转到 (6)
    /* 判断如果 $y$  为0，则输出错误提示，并转到 (6) */
(4) else  $z \leftarrow x/y$  /* 否则计算 $x \div y$ ，并赋值给 $z$  */
(5) 输出 $z$           /* 输出 $z$ 的值 */
(6) End            /* 算法伪代码结束 */
```

5.3 算法

3. 算法表示方法—计算机高级语言表示

高级语言就是用计算程序设计语言直接表达算法，是可以直接在计算机上运行的源程序。高级语言描述的算法优点是**严格、准备**，但语言**细节过多**。

【例】输入 x 、 y ，计算 $z=x \div y$ ，输出 z 的计算机c语言描述形式。

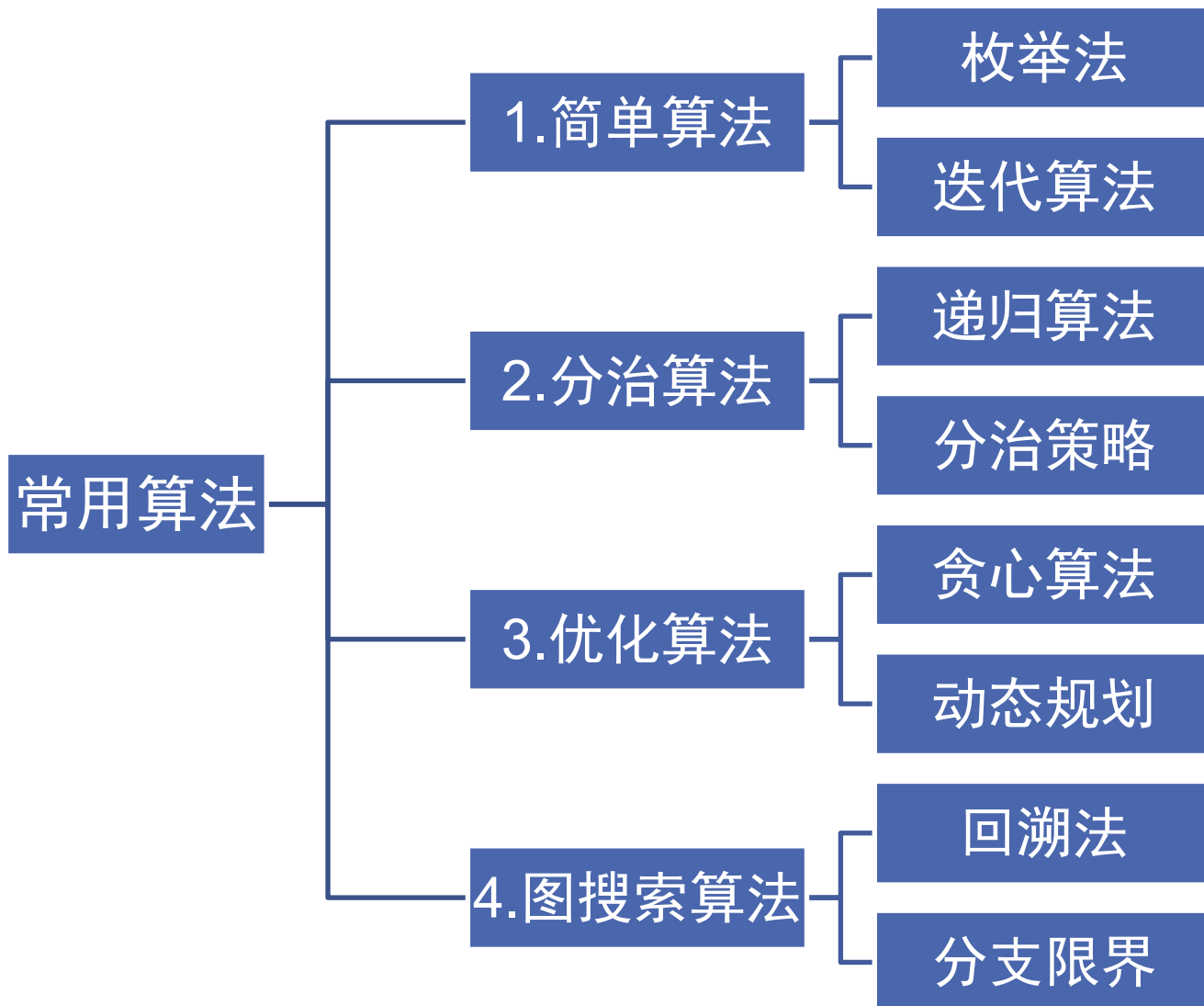
```
float div(float x, float y)
{
    float z;
    if(y==0)
        printf("Error!\n");
    else
        z=x/y;
    return z;
}
```

课堂测试

算法是解决问题的方法或步骤，描述算法的方式有多种，下列方式不能描述算法的是（ **B** ）。

- A. 自然语言
- B. 表格方式
- C. 程序设计语言
- D. 程序设计语言和自然语言相结合

5.4 常用算法



5.3 算法

1. 递归算法

递归的定义： **程序调用自身的编程技巧**称为**递归**, 是函数自己调用自己。如两面镜子相互之间近似平行摆放, 镜中嵌套的图像是以无限递归的形式出现。

递归分为两个阶段:

- **递推:** 把复杂的问题的求解推到比**原问题简单一些**的问题的求解;
- **回归:** 当获得最简单情况后, 逐步返回, 依次得到复杂的解。

递归的特点:

- **优点:** 代码简洁清晰
- **缺点:** 空间消耗要比非递归代码要大很多, 如果递归深度太大, 可能会造成栈溢出。

5.3 算法

1. 递归算法

阶乘函数：阶乘函数可递归地定义为：

$$n! = \begin{cases} 1 & n = 0 \\ n(n-1)! & n > 0 \end{cases}$$

边界条件

递归方程

边界条件与递归方程是递归函数的两个要素，递归函数只有具备了这两个要素，才能在有限次计算后得出结果。

```
int f(n)
{
    if(n==0) return 1;
    return n*f(n-1);
}
```

5.3 算法

1. 递归算法

Fibonacci数列：无穷数列1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ..., 被称为斐波拉契数列。它可以递归地定义为：

$$F(n) = \begin{cases} 1 & n = 0 \\ 1 & n = 1 \\ F(n-1) + F(n-2) & n > 1 \end{cases}$$

递归基

递归表达式

第n个斐波那契数可递归地计算如下：

```
int fib (int n)
{
    if (n <= 1) return 1;
    return fib (n-1) + fib(n-2);
}
```

- 递归表达式比一般表达式简单
- 递归表达式不变，仅是参数在缩小，直到递归基

5.3 算法

1. 递归算法

问：老奶奶进城买的某个智慧农场生产的农产品需要 m 元。由于近视眼，用的是Nokia老人机，没有微信支付。此时，她身上有 $a, b, c, \dots, N$ 种货币，问能否凑出所需的 m 元钱。

问：假设有2, 3, 5, 7四种货币各一个，能否凑出10块钱？

方案

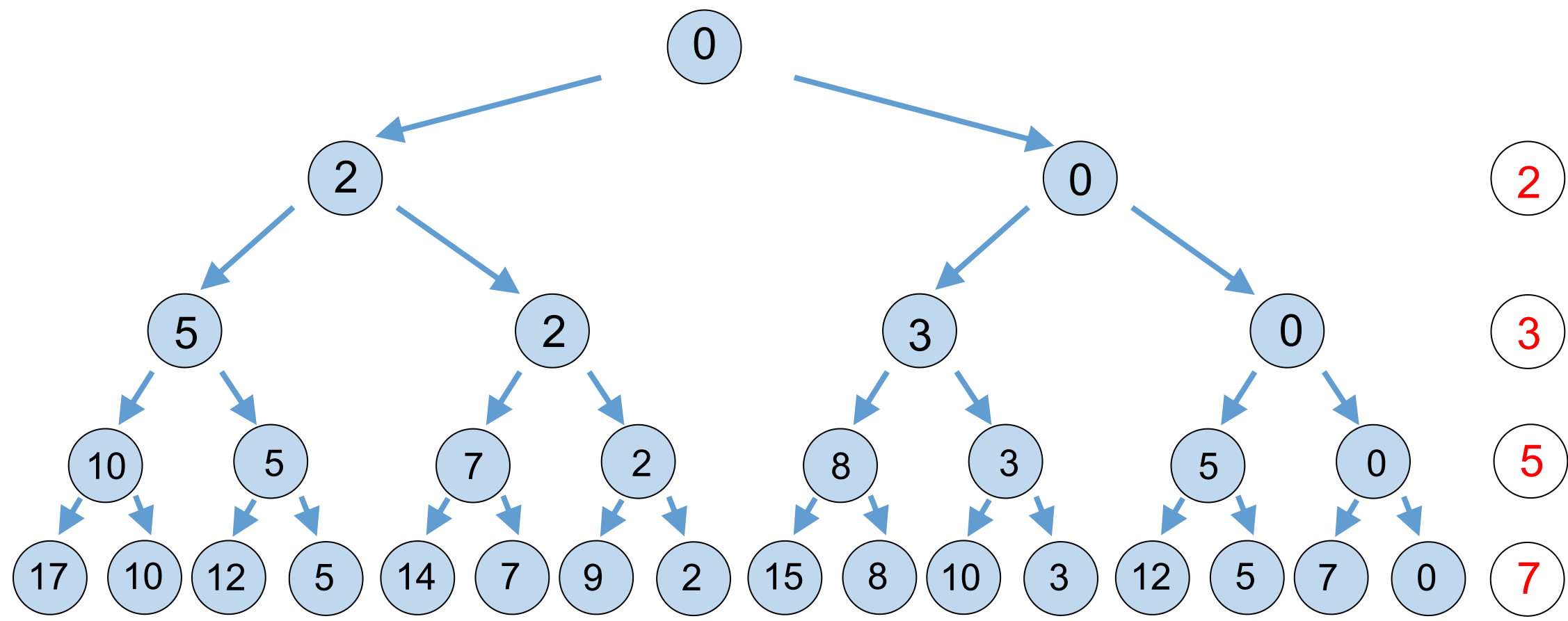
- 把{2, 3, 5, 7} 看成一个集合
- 每一种凑钱的方法都是它的一个子集。
- 遍历集合所有的子集。

N 个元素共 2^N 个子集，

目标就是想办法遍历这 2^N 个子集，即二叉树展开。

5.3 算法

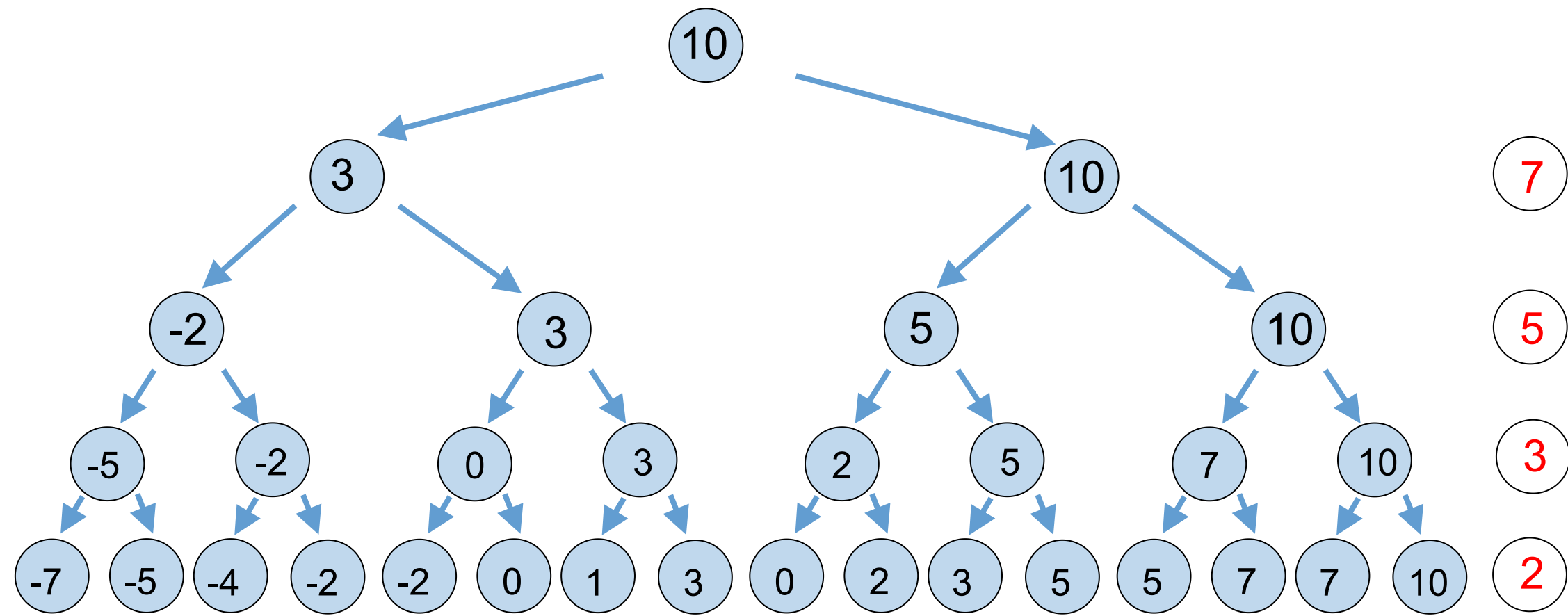
二叉树展开：树的节点表示已经凑的钱数。2，3，5，7四种货币，凑钱时，每种货币都有选（左子树）和不选（右子树）的可能。



完全二叉树展开，叶子节点就是所有{2，3，5，7}凑出钱的结果。

5.3 算法

二叉树展开：**树的节点表示我还要凑的钱数**。2, 3, 5, 7四种货币，凑钱时，每种货币都有**选（左子树）**和**不选（右子树）**的可能。



阶乘递归：计算n的阶乘变成计算n-1的阶乘（仅一个值）
要凑10块钱：变成了要凑3块钱和要凑10块钱（两种可能）

5.3 递归代码

老奶奶凑钱递归算法实现

```
def process(a, aim, index, curr):
```

```
# a          表示拥有的钱的列表  
# aim        要凑的目标  
# index      二叉树的层数  
# curr       已经凑的钱数
```

```
    return process(a, aim, index+1, curr+a[index]) | process(a, aim, index+1, curr)
```

5.3 递归代码

老奶奶凑钱递归算法实现

```
def process(a, aim, index, curr):  
    if aim == curr:  
        print("success")  
        return True  
    if index == len(a):  
        print("No success")  
        return False  
    return process(a, aim, index+1, curr+a[index]) | process(a, aim, index+1, curr)  
  
if __name__ == '__main__':  
    a=[2,3,5,7]  
    aim=10  
    process(a,aim, index: 0, curr: 0)
```

a 表示拥有的钱的列表
aim 要凑的目标
index 二叉树的层数
curr 已经凑的钱数

G:\ProgramFiles\python311\python.exe

success

No success

No success

No success

No success

No success

No success

No success

No success

success

No success

No success

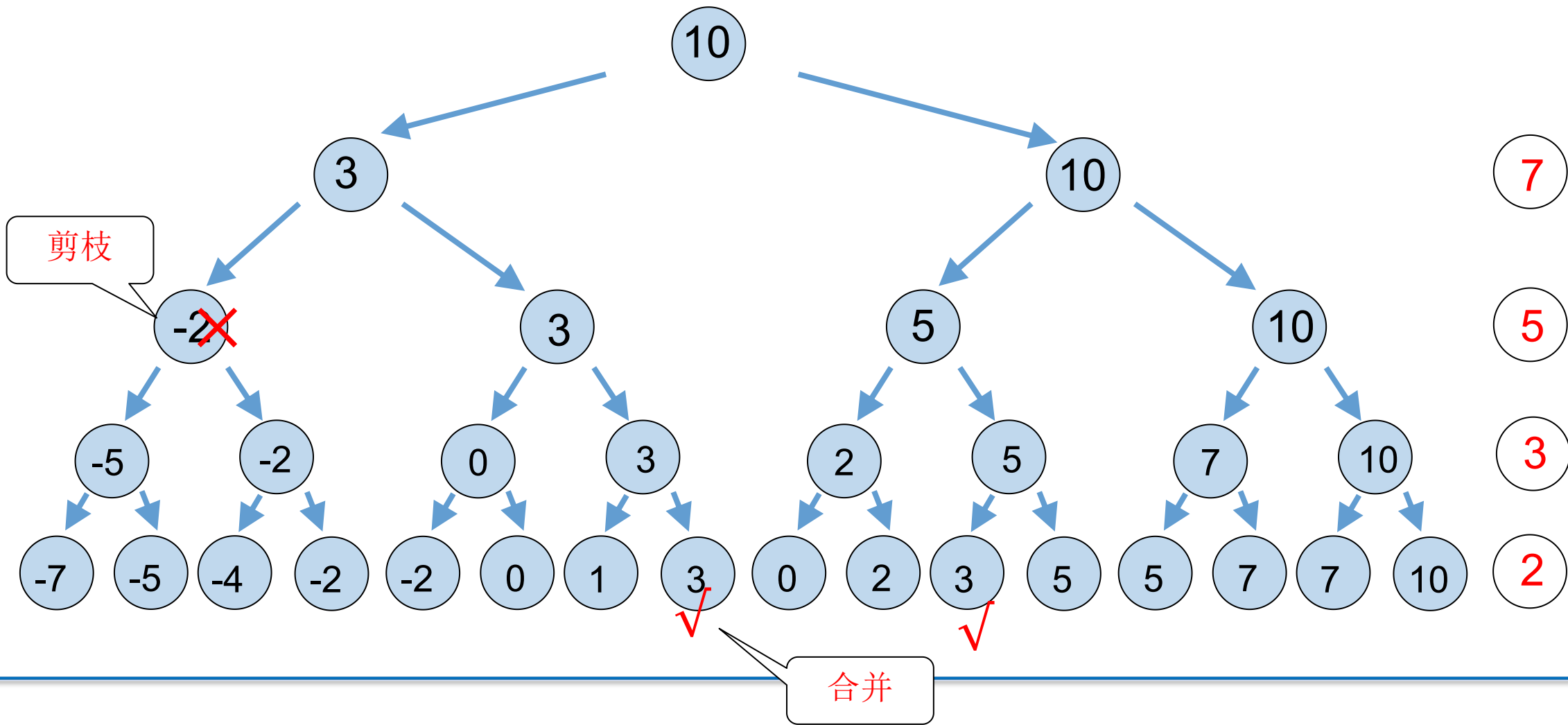
No success

No success

No success

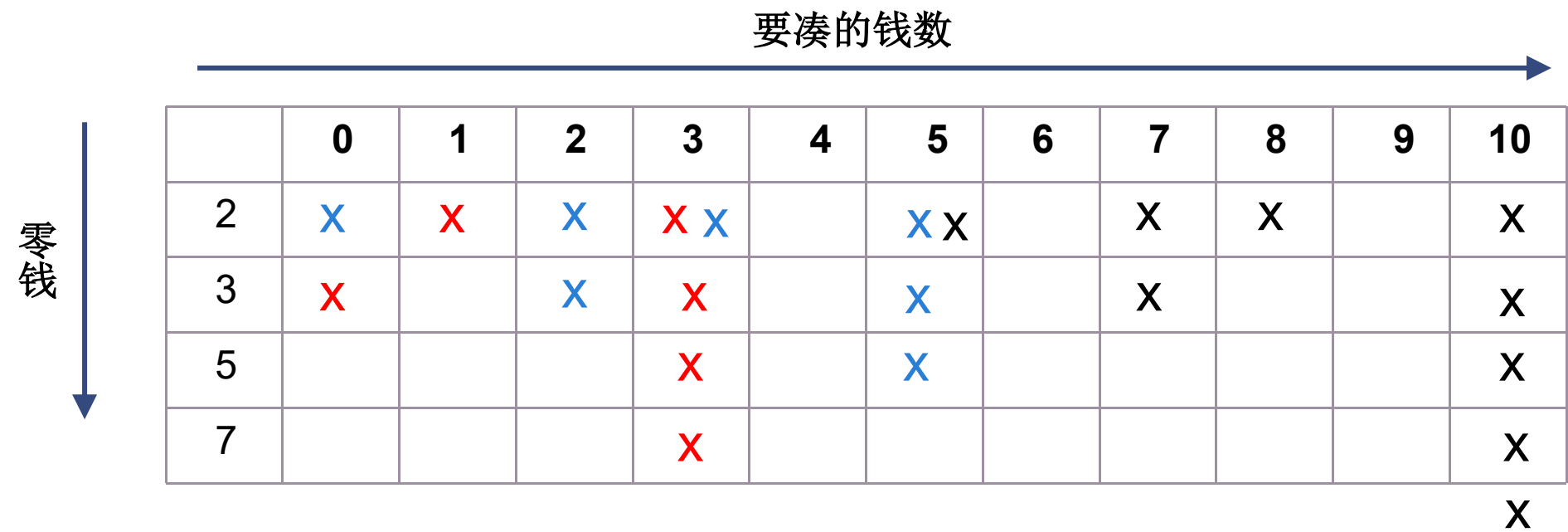
5.3 算法

优化：二叉树展开时间复杂度为 $O(2^N)$ 。



5.3 算法

动态规划：老奶奶的目标是凑10块钱。x表示用这行以上的元素集，凑出列所在的钱。



- 二叉树展开的时间复杂度为 $O(2^n)$ ，表格是 $O(n^2)$
- 动态规划最笨的方法就是递归。从某种程度上说，递归就是一种遍历，就是一种穷尽。

动态规划：代码实现

G:\ProgramFiles\python311\python.exe

```
[[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1],
 [0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1],
 [0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1],
 [1, 0, 1, 1, 0, 1, 0, 1, 0, 0, 1],
 [1, 1, 1, 1, 0, 1, 0, 1, 1, 0, 1]]
```

[illegible]

5.3 算法

2. 枚举法

枚举法又称穷举法，基本思想是根据题目的部分条件确定答案的大致范围，并在此范围内**对所有可能的情况逐一验证**，直到全部情况验证完毕。枚举法也称为穷举法。

特点：

- ① 枚举法是一种几乎什么问题都能解决的一般性方法。
 - ② 效率低下，但是可用于求解一些小规模的问题实例。如果解决问题实例不多，而枚举法可用一种可接受的速度对问题求解，那么花时间去设计一个更高效的算法是得不偿失的。
-

5.3 算法

2. 枚举法

【例】子数整数问题。对于一个五位数 $abcde$ ，可将其拆分为三个子数： abc 、 bcd 和 cde 。

$$\text{sub1}=abc \quad \text{sub2}=bcd \quad \text{sub3}=cde$$

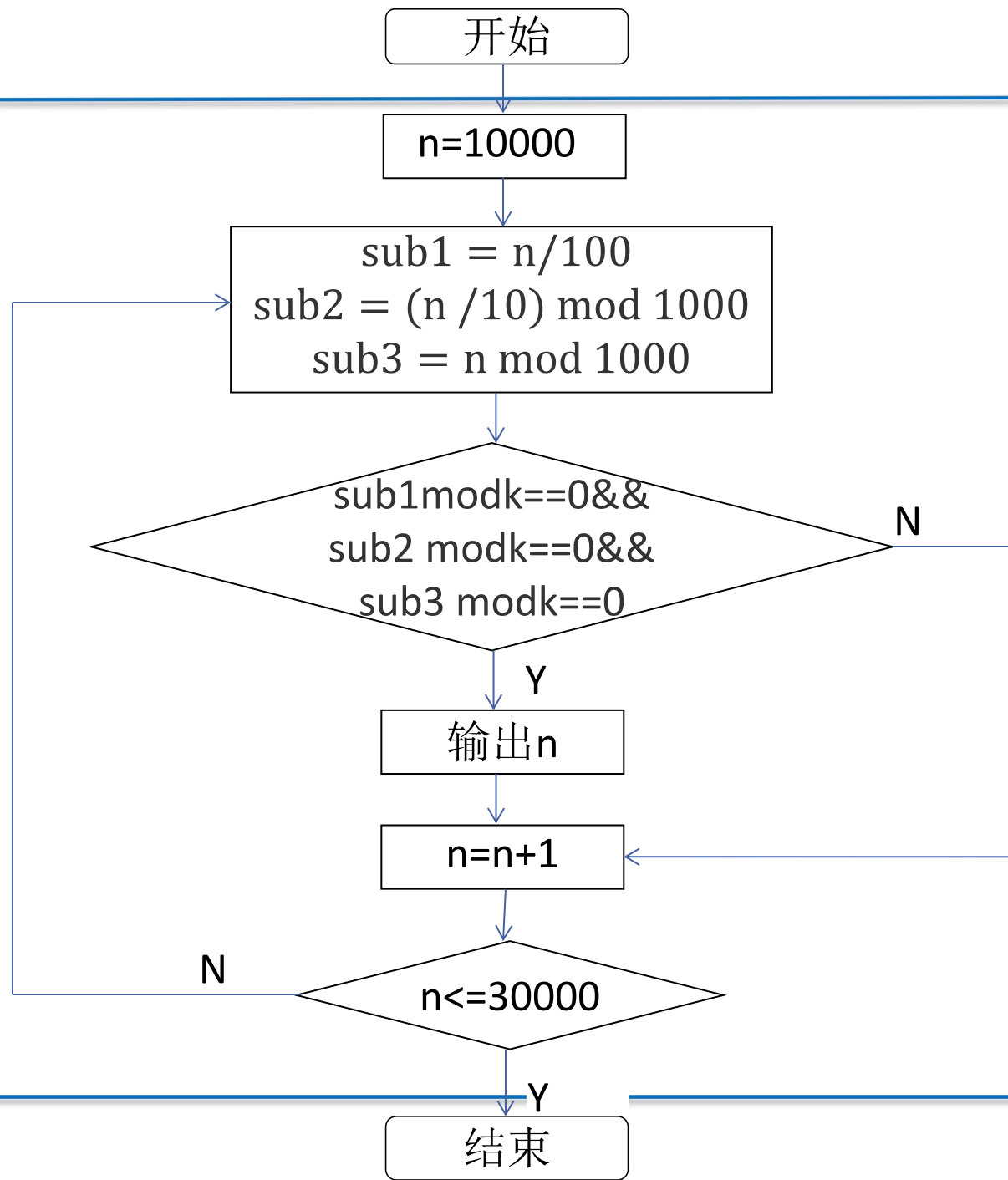
现在给定一个正整数 K ，要求求出10000到30000之间所有满足下述条件的五位数，条件是这些五位数的三个子数 sub1 ， sub2 ， sub3 都可被 K 整除。

5.3 算法

2. 枚举法

利用枚举法解决这个问题，即在有限集合10000到30000之间，找出满足条件的每个数。步骤如下：

1. $n=10000$
2. 对 n 通过整除和求余运算得到子数
 $sub1=n/100$
 $sub2=(n/10) \bmod 1000$
 $sub3=n \bmod 1000$
3. 判断 $sub1$ 、 $sub2$ 、 $sub3$ 是否都可被 k 整除。
4. 如果满足输出 n ， $n++$ ；
5. 如果不能整除 k ， $n++$ ；
6. If $n \leq 30000$, goto 步骤2, else 结束。



5.3 算法

3. 迭代法

- 迭代法也称“辗转法”，其实就是不断地**用旧的变量值，递推计算新的变量值。利用变量的原值推算出变量的一个新值**。如果递归是自己调用自己的话，迭代就是A不停的调用B。
- **优点**：迭代效率高，运行时间只因循环次数增加而增加；
- **缺点**：1) 不容易理解；2) 代码不如递归简洁；3) 编写复杂问题时困难。

5.3 算法

3. 迭代法

□ 例：验证谷角猜想。

日本数学家谷角静夫在研究自然数时发现了一个奇怪现象：对于任意一个自然数 n ，若 n 为偶数，则将其除以2；若 n 为奇数，则将其乘以3，然后再加1。如此经过有限次运算后，总可以得到自然数1。

算法分析：

- 1) **确定迭代变量**, n 就是迭代变量；
- 2) **确定迭代关系**，有两种迭代关系：一种是当 n 为偶数时 $n=n/2$ ，一种是当 n 为奇数时 $n=n*3+1$ ；
- 3) **迭代控制**，通常就是找到循环结束条件，本例就是当 $n=1$ 结束，验证猜想成功。

5.3 算法

3. 迭代法

谷角猜想算法实现步骤:

- 1) 输入 n ;
- 2) if n 为偶数, $n=n/2$; 否则执行3) ;
- 3) $n=n*3+1$;
- 4) 判断 n 是否为1, if n 等于1, 程序结束; 否则goto 2) 。

```
Guess( )
{
    int n;
    scanf("%d ", &n);
    while (n != 1)
    {
        if(n%2==0)
             $n=n/2$ ;
        else
             $n=3*n+1$ ;
    }
    printf("%d",n) ;
}
```

5.3 算法

4 分治算法

□ **基本思想**： 对一个规模为 n 的问题，若该问题可以容易地解决(比方说规模较小)，则直接解决；否则将其**分解为 k 个规模较小的子问题**，这些子问题**相互独立且与原问题形式相同**。分治策略递归地分解这些子问题，然后将各个子问题的解合并得到原问题的解。

□ **基本步骤**：

- (1) **分解**，将要解决的问题划分成若干规模较小，**相互独立，与原问题同类的子问题**；
- (2) **求解**，当子问题规模划分得足够小时，用较简单的方法解决；
- (3) **合并**，按原问题的要求，将子问题的解逐层合并构成原问题的解。

分治法与递归算法

分治和递归常被比喻为一对孪生兄弟，因为它们在算法设计中往往紧密相连。

□ 分治策略是方法论

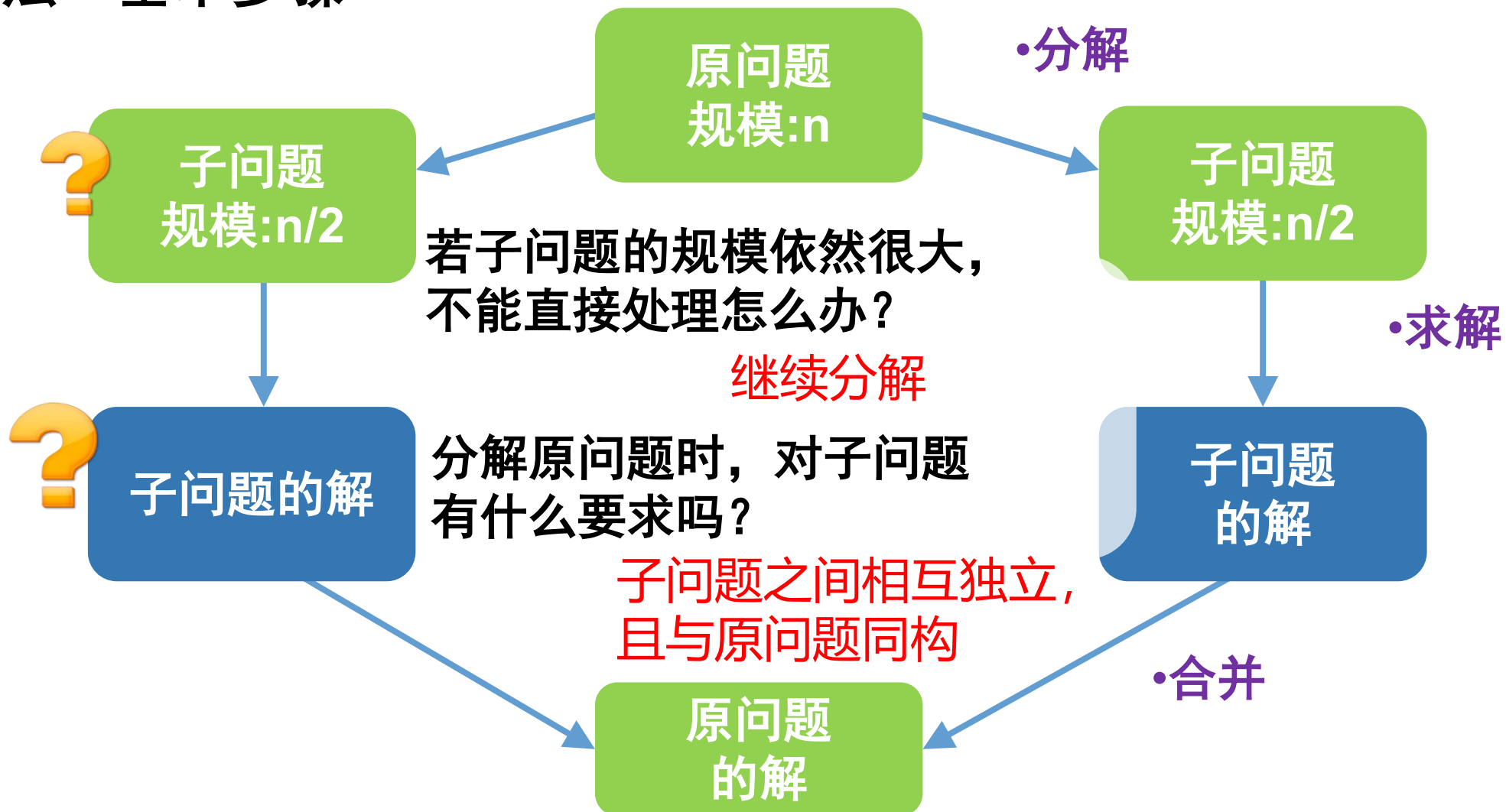
1. 分治 (Divide and Conquer) 是一种重要的**算法设计策略**，它将一个复杂的大问题划分为几个相同或相似的小问题，然后分别解决这些小问题，最后将各个小问题的解组合起来得到原问题的解。
2. 分治的关键步骤包括三个部分：**Divide** (划分问题)、**Conquer** (解决子问题) 和 **Combine** (合并子问题的解)。

□ 递归算法是实现手段

1. 递归是一种**编程技术**，其中函数在其定义中直接或间接地调用自身来解决问题。
 2. 实现分治策略时，递归是一个强有力的工具，因为它能够自然地表达出“将一个问题拆分成多个相似的小问题，然后通过解决小问题来解决大问题”的过程。
-

5.3 算法

分治法—基本步骤



分治法案例

【例】Kitty猫基因编码问题：某教授告诉大家地球上Kitty猫的基因长度都是2的正整数次幂 2^k ($k \leq 10$)，全是由两种不同的基因单元组成的。这两种不同的基因单元分别记成0和1，于是Kitty猫基因可以写成一个01串表达式s。该编码规则是不断地按照下面的规则对Kitty猫基因01串表达式s进行改写，直至最终被改写成只含有字符“A”、“B”、“C”的符号串。

$$T(s) = \begin{cases} A & \text{若S串全为0} \\ B & \text{若S串全为1} \\ CT(s_1)T(s_2) & \text{否则把s分成两个等长串是s1和s2} \end{cases}$$

$$\begin{aligned} & T(0\ 1\ 0\ 0\ 1\ 0\ 1\ 1) \\ &= C\ T(0\ 1\ 0\ 0)\ T(1\ 0\ 1\ 1) \\ &= C\ C\ T(0\ 1)\ T(0\ 0)\ C\ T(1\ 0)\ T(1\ 1) \\ &= C\ C\ C\ T(0)\ T(1)\ A\ C\ C\ T(1)\ T(0)\ B \\ &= C\ C\ C\ A\ B\ A\ C\ C\ B\ A\ B \end{aligned}$$

5.3 算法

4. 分治法案例求解

(1) 问题分析:

基本思想是将 n 个01基因分成个数相同的两部分 s_1 和 s_2 。如果 s_1 全为0, 编码为A; 如果全为1, 编码为B; 否则将 s_1 (s_2) 分成等长串; 重复此过程, 直到编码结束为止。

(2) 算法实现 (自然语言)

编码算法实质上是不断地将01字符串进行对半分割, 并检查每个字符串, 对其进行编码或继续分割, 直到不能分割为止。

Step1: if s 串全0 编码A;

Step2: if s 串全1 编码B;

Step3: if s 串不满足上面两种情况 将 s 串分解为相等子串 s_1 和 s_2 ;

Step4: 对于 s_1 和 s_2 重复执行Step1- Step3, 直至分割结束。

Step5: return s 串编码。

分治法案例求解

```
def kitty_gene_encoding(s):  
    # 判断字符串是否全为0或全为1  
    if s == '0' * len(s):  
        return 'A'  
    elif s == '1' * len(s):  
        return 'B'  
    # 将字符串等分为两个子串  
    half = len(s) // 2  
    left_str = s[:half]  
    right_str = s[half:]  
    # 对子串递归编码  
    left_code = kitty_gene_encoding(left_str)  
    right_code = kitty_gene_encoding(right_str)  
    # 拼接编码并加上'C'  
    return 'C' + left_code + right_code  
  
if __name__ == '__main__':  
    input_gene = "01001011" # 输入示例  
    encoded_gene = kitty_gene_encoding(input_gene)  
    print(encoded_gene) # 输出为 'CCCABACCBAB'
```

(2) 算法实现（自然语言）

编码算法实质上是不不断地将01字符串进行对半分割，并检查每个字符串，对其进行编码或继续分割，直到不能分割为止。

Step1: if s串全0 编码A;

Step2: if s串全1 编码B;

Step3: if s串不满足上面两种情况 将s串分解为相等子串 s_1 和 s_2 ;

Step4: 对于 s_1 和 s_2 重复执行Step1- Step3, 直至分割结束。

Step5: return s串编码。

5.3算法—贪心算法

5. 贪心算法

贪心算法又称贪婪算法，其**基本思想**是在解决问题时，遵循“每一步都采取当前状态下最好或最优的选择”的策略（**即局部最优**），而不考虑长远的、整体的最优性。换言之，贪心算法在求解过程中，仅依据当前已有的信息来作出决策，希望以此步步为营的方式最终能够得到全局最优解。因此，对于那些**局部最优解**无法确保全局最优的问题，贪心算法可能只能得到**近似解或者局部最优解**。

贪心算法可以求解很多的优化问题，在农业优化问题方面，如最优施肥，最优灌溉，最优货物装载，路线最优规划等问题都可以使用贪心算法来解决。

5.3算法—贪心算法

贪心算法案例求解

【例】农机安排问题。

假设某区域有1台农机，有n块地块 $A = \{1,2, \dots, n\}$ 等待收割，所有地块都需要使用同一台农机，每地块均有一个收割开始和结束的时间。问如何安排农机，才能让**农机收割完尽量多的地块**。

假设待安排的11个地块收割的**开始时间s[i]**和**结束时间f[i]**如下表：

	1	2	3	4	5	6	7	8	9	10	11
s[i]	1	3	0	5	3	5	6	8	8	2	12
f[i]	4	5	6	7	8	9	10	11	12	13	14

5.3算法—贪心算法

贪心选择策略是**使剩余的可安排时间段极大化**，以便安排尽可能多的地块。因此对收割地块**依据其结束时间**进行非减序排列，结束时间越早的地块优先使用农机，这样可以安排尽可能多的地块。

	1	2	3	4	5	6	7	8	9	10	11
s[i]	1	3	0	5	3	5	6	8	8	2	12
f[i]	4	5	6	7	8	9	10	11	12	13	14

贪心算法实现：将第一个地块结束时间f[1]与后面活动开始时间s[2]相比较，若 $s[2] < f[1]$ 则继续比较，直到 $s[2] > f[1]$ ，选中此地块，以此类推。**最后选出符合条件地块1，4，8，11。**

5.3 算法—贪心算法

贪心算法—举例

`fields`表示地块收割的起始时间和结束时间的列表

```
def greedy_schedule(fields):# fields 是一个列表, 其中每个元素是一个包含起始时间和结束时间的元组 (si, fi)
    # 按照结束时间 fi 升序排序地块
    sorted_fields = sorted(fields, key=lambda x: x[1])

    schedule = []
    current_time = 0

    for field in sorted_fields:
        si, fi = field
        if si >= current_time: # 可以安排此地块
            schedule.append(field)
            current_time = fi # 更新当前时间
    return schedule

if __name__ == '__main__':
    # fields里是具体的地块起止时间
    fields =[[1,4], [3,5], [0,6], [5,7], [3,8], [5,9], [6,10], [8,11], [8,12], [2,13], [12,14]]
    print(greedy_schedule(fields))
```

```
G:\ProgramFiles\python311\python.exe
```

```
[[1, 4], [5, 7], [8, 11], [12, 14]]
```

```
Process finished with exit code 0
```

本章小结

1. 三种科学思维
2. 计算思维的定义
3. 计算思维的5大特征
4. 问题以及问题求解
5. 算法的定义
6. 算法的特征 (4个)
7. 算法的表示方法